

 Latest updates: <https://dl.acm.org/doi/10.1145/3725399>

RESEARCH-ARTICLE

Published: 18 June 2025

[Citation in BibTeX format](#)

DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter

MENGXU JIANG, Chinese University of Hong Kong, Hong Kong, Hong Kong

ZHI YANG, Huazhong University of Science and Technology, Wuhan, Hubei, China

FANGYUAN ZHANG, Chinese University of Hong Kong, Hong Kong, Hong Kong

GUANHAO HOU, Chinese University of Hong Kong, Hong Kong, Hong Kong

JIEMING SHI, The Hong Kong Polytechnic University, Hong Kong, Hong Kong, Hong Kong

WENCHAO ZHOU, Alibaba Group Holding Limited, Hangzhou, Zhejiang, China

[View all](#)

Open Access Support provided by:

Huazhong University of Science and Technology

Chinese University of Hong Kong

The Hong Kong Polytechnic University

Alibaba Group Holding Limited

DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter

MENGXU JIANG, The Chinese University of Hong Kong, Hong Kong

ZHI YANG, Huazhong University of Science and Technology, China

FANGYUAN ZHANG*, The Chinese University of Hong Kong, Hong Kong

GUANHAO HOU, The Chinese University of Hong Kong, Hong Kong

JIEMING SHI, The Hong Kong Polytechnic University, Hong Kong

WENCHAO ZHOU, Alibaba Cloud, China

FEIFEI LI, Alibaba Cloud, China

SIBO WANG[†], The Chinese University of Hong Kong, Hong Kong

Recent advancements in AI have enabled models to map real-world entities, such as product images, into high-dimensional vectors, making approximate nearest neighbor search (ANNS) crucial for various applications. Often, these vectors are associated with additional attributes like price, prompting the need for range-filtered ANNS where users seek similar items within specific attribute ranges. Naive solutions like pre-filtering and post-filtering are straightforward but inefficient. Specialized indexes, such as SeRF, SuperPostFiltering, and iRangeGraph, have been developed to address these queries effectively. However, these solutions do not support dynamic updates, limiting their practicality in real-world scenarios where datasets frequently change.

To address these challenges, we propose *DIGRA*, a novel dynamic graph index for range-filtered ANNS. *DIGRA* supports efficient dynamic updates while maintaining a balance among query efficiency, update efficiency, indexing cost, and result quality. Our approach introduces a dynamic multi-way tree structure combined with carefully integrated ANNS indices to handle range filtered ANNS efficiently. We employ a lazy weight-based update mechanism to significantly reduce update costs and adopt optimized choice of ANNS index to lower construction and update overhead. Experimental results demonstrate that *DIGRA* achieves superior trade-offs, making it suitable for large-scale dynamic datasets in real-world applications.

CCS Concepts: • **Information systems** → **Retrieval models and ranking**.

Additional Key Words and Phrases: Range-Filtering Approximate Nearest Neighbor Search; Vector Database; Dynamic Indexing

ACM Reference Format:

Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guan hao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 148 (June 2025), 26 pages. <https://doi.org/10.1145/3725399>

*Work done during an internship at Alibaba Group.

[†]Sibo Wang is the corresponding author.

Authors' Contact Information: Mengxu Jiang, mxjiang@se.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong; Zhi Yang, yazhi201@hust.edu.cn, Huazhong University of Science and Technology, China; Fangyuan Zhang, fzhang@se.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong; Guan hao Hou, ghhou@se.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong; Jieming Shi, jieming.shi@polyu.edu.hk, The Hong Kong Polytechnic University, Hong Kong; Wenchao Zhou, zwc231487@alibaba-inc.com, Alibaba Cloud, China; Feifei Li, lifefeifei@alibaba-inc.com, Alibaba Cloud, China; Sibo Wang, swang@se.cuhk.edu.hk, The Chinese University of Hong Kong, Hong Kong.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2836-6573/2025/6-ART148

<https://doi.org/10.1145/3725399>

1 Introduction

In today's landscape of big data and AI, effectively managing high-dimensional vectors is essential for modern applications like recommendation engines [21], image retrieval [15], and natural language processing [10, 26]. Machine learning models convert various data types into vector representations, enabling real-world applications to perform queries within vector spaces [16, 20, 27, 31]. A key task in this domain is identifying the nearest vector to a given query vector, which is critical for functionalities such as object recognition [37] and personalized content delivery [8].

However, as vector dimensionality increases, traditional exact search methods become inefficient due to the curse of dimensionality, making approximate nearest neighbor search (ANNS) a more practical solution. ANNS techniques enable the efficient retrieval of vectors similar to a query vector, supporting large-scale vector search operations. Vector databases like Milvus [33] and AnalyticDB [35] enhance search precision by integrating ANNS with attribute-based filtering. For example, e-commerce platforms can provide product recommendations based on image similarity while also applying filters for price or user age [28, 33, 39]. To handle high-dimensional vectors and range-filtered queries effectively, modern vector databases utilize various algorithms to accelerate ANNS with range filters. Although combining ANNS with range filtering is crucial for many applications, existing methods often fall short in addressing this challenge efficiently.

Common approaches to manage such queries involve pre-filtering or post-filtering techniques. Pre-filtering filters the dataset based on attribute criteria and then conducts nearest neighbor search. Post-filtering, conversely, conducts ANNS first and then filters the results according to the attribute constraints. These methods are straightforward to implement and support dynamic updates to the dataset, making them compatible with database systems and widely adopted in various vector databases [33, 35, 39]. However, they exhibit sub-optimal performance on large datasets. Pre-filtering can be time-consuming due to the need for extensive preliminary scans, while post-filtering may miss many similar objects, leading to delays in query completion.

To gain better query efficiency for ANNS with range filters, recent research has designed specialized algorithms that construct dedicated indexes to enable efficient query processing. One such method is SeRF [40], which addresses ANNS with range filters by utilizing Hierarchical Navigable Small World (HNSW) graphs. HNSW is a popular index structure for ANNS that organizes data points into a hierarchical, multi-layered graph, allowing efficient navigation through high-dimensional spaces by connecting vertices (vectors) via edges based on proximity. This structure facilitates fast approximate searches by traversing from higher levels of the hierarchy down to the most relevant vertices. In SeRF, the approach involves compressing $O(n^2)$ HNSW graphs—each corresponding to different range filters—into a single, unified graph. This method theoretically incurs a space overhead of up to $O(n^2M)$, where M is a parameter in HNSW that controls the maximum number of connections per vertex, significantly impacting memory usage. To mitigate this substantial space requirement, SeRF employs a compression technique called MaxLeap, which reduces the number of edges by allowing longer-range connections. However, this compression leads to the loss of crucial graph structure information, resulting in suboptimal query performance. Additionally, because the construction of SeRF must follow a specific attribute order, inserting or deleting objects affects many subsequent HNSW constructions, rendering it incapable of supporting dynamic updates.

SuperPostFiltering [12] addresses the efficiency issue by creating graph indices for various predefined range combinations. During the query phase, it selects the appropriate pre-constructed index to answer the query. While this method can improve query efficiency, it generates a large number of redundant graph indices, leading to significant memory overhead. Moreover, when objects are inserted or deleted from the dataset, the set of maintained ranges changes substantially,

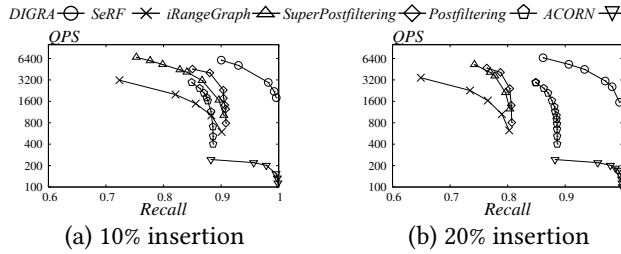


Fig. 1. Impact of insertion on SIFT.

making it impractical to support dynamic updates. A more recent method, iRangeGraph [38], utilizes a segment tree to manage indices, which reduces overlaps between graph indices and achieves both stable query performance and reduced space overhead. However, segment trees are inherently static structures that cannot efficiently handle updates. As noted in [38], iRangeGraph does not support dynamic insertion or deletion of objects, limiting its applicability in environments where datasets change over time.

In real-world scenarios, vector databases are dynamic, with new data frequently added and old data removed. For instance, as illustrated in Fig. 1(a) (resp. Fig. 1(b)), starting with 90% (resp. 80%) vectors from a base dataset of 1 million SIFT vectors, the addition of the other 10% (resp. 20%) vectors significantly degrades the search efficiency of static indexes like SeRF, iRangeGraph, and SuperPostfiltering when handling range-filtered ANNS. Specifically, to achieve a high recall rate of 90%, the Queries Per Second (QPS) performance deteriorates markedly. This degradation becomes more pronounced with the insertion of 20% new data. Results on other datasets are similar, as seen in Sec. 6. In fact, when a substantial portion of new elements is added, relying on the static index built on the original dataset leads to significant drops in recall. While one might consider periodically rebuilding the index to handle updates, this approach is impractical due to the high computational costs and resource consumption involved in rebuilding large-scale indices. Moreover, determining the optimal frequency for rebuilding is challenging: Rebuilding too often incurs excessive overhead, while doing it infrequently leads to outdated indices and degraded query performance.

To address this limitation, we propose **DIGRA**¹, an effective index that achieves a superior trade-off among query efficiency, update efficiency, indexing cost, and query result quality (in terms of recall). As we can see from Figure 1, our DIGRA, which supports dynamic index updates, consistently maintains high query efficiency and high recall, even as new data is added, underscoring the importance of supporting index updates.

Achieving a balance among all four aspects—query efficiency, update efficiency, indexing cost, and result quality—is challenging because previous methods struggle with updates due to their static nature or costly rebuilding processes when data changes. Specifically, existing solutions face difficulties because they: (i) encode the graph index and range information using static structures that are not amenable to efficient updates; (ii) build multi-level graph index structures that necessitate complete or partial rebuilding when the dataset changes, incurring significant update costs. To overcome the challenges, DIGRA includes the following innovative designs:

Dynamic multi-way tree structure. We employ a dynamic multi-way tree T to organize the index based on the attribute ϕ of range filtering. Each node in T uses ϕ as the key and stores mapped vector IDs as values. By utilizing split and merge operations similar to those in B-trees, we can adjust the tree structure efficiently during insertions and deletions. This dynamic approach allows the index to accommodate data updates seamlessly.

¹Dynamic Graph Index for Range-filtered ANN

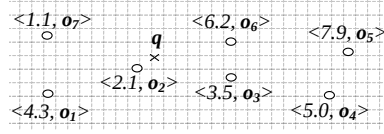


Fig. 2. Example of an object set

Efficient range query handling. For any arbitrary range $[\ell, r]$, we can efficiently identify at most two nodes in T in $O(\log n)$ time such that all vectors with attribute values within this range are stored in the subtrees of these two nodes. This property enables us to retrieve vectors matching the range filter without scanning irrelevant parts of the tree, thereby enhancing query efficiency.

Integration of ANNS indices at tree nodes. To accelerate approximate nearest neighbor search (ANNS) within the filtered ranges, we build an ANNS index G_u (e.g., a Navigable Small World (NSW) or a Hierarchical Navigable Small World (HNSW) graph index) at each internal node u of T . This index is constructed over the vectors stored in the subtree rooted at u . During a range-filtered ANNS query, we first locate the relevant nodes u and v , perform ANNS searches using G_u and G_v , and then merge the results to obtain the top- k answers. This hierarchical indexing leverages the tree structure to improve search performance.

Lazy weight-based update mechanism. Maintaining the ANNS indices G_u at each node u in the multi-way tree T during dynamic updates presents a significant challenge, especially when split and merge operations modify the number of elements in affected subtrees. For instance, splitting a node can impact numerous descendants, necessitating the costly rebuilding of their ANNS indices. To address this, we introduce a *lazy weight-based update mechanism* that reduces the update cost to an amortized $O(c_{upd} \cdot \log n)$ per update, where c_{upd} represents the time complexity of inserting an object into an ANNS index. Additionally, we employ a background rebuilding strategy to eliminate amortization effects, ensuring consistent update performance without degrading query efficiency. This approach effectively balances the need for dynamic updates with the preservation of high query performance and recall quality.

Optimized choice of ANNS index. While it might initially seem advantageous to use HNSW graphs at each node of our multi-way tree—given HNSW’s typically higher Queries Per Second (QPS) compared to other graph-index methods that support update like NSW while achieving similar recall rates [34], we recognize that HNSW’s primary strength lies in its inherent hierarchical structure. Since our multi-way tree already provides a hierarchical organization, incorporating HNSW would result in redundant layering of hierarchies. Therefore, we opt for NSW graphs as the ANNS indices at each node for several reasons. First, NSW graphs have significantly lower construction and update costs, being up to an order of magnitude less computationally intensive to build and maintain compared to HNSW graphs [34], thereby reducing overall indexing and update overhead. Second, the flat structure of NSW graphs aligns seamlessly with the hierarchical organization of the multi-way tree, facilitating efficient searches within subtrees without introducing unnecessary complexity.

In summary, we make the following contributions.

- By combining the multi-way tree and NSW structures, DIGRA achieves efficient range-filtered searches with high recall, without relying on redundant graph indices.
- With the structural design of DIGRA, we introduce an efficient lazy weight-based update mechanism, ensuring real-time index maintenance without the need for complete index rebuilding.
- We provide a rigorous theoretical analysis showing that the update overhead of DIGRA is only $O(\log n)$ times larger than that of inserting a point in a vanilla NSW, which translates to at least an $O(n/\log n)$ reduction in update costs compared to traditional approaches.

- Extensive experiments on real-world high-dimensional vector datasets demonstrate that DIGRA maintains excellent query performance, while reducing update overhead by five orders of magnitude compared to methods that require index rebuilding.

2 PRELIMINARIES

2.1 Problem Definition

Given a set X and a distance function $\delta : X \times X \rightarrow \mathbb{R}$, the pair (X, δ) is called a metric space if, for all $x, y, z \in X$, (i) $\delta(x, y) \geq 0$, with $\delta(x, y) = 0$ if and only if $x = y$; (ii) $\delta(x, y) = \delta(y, x)$; and (iii) $\delta(x, y) \leq \delta(x, z) + \delta(y, z)$. For example, if $X = \mathbb{R}^d$ is the set of all d -dimensional real vectors and δ is the Euclidean distance, then (X, δ) is a Euclidean space. In a metric space (X, δ) , the nearest neighbor search (NNS) problem, which has numerous real-world applications, is defined as follows:

Definition 2.1 (Nearest Neighbors Search (NNS)). Let (X, δ) be a metric space. Given a set $O \subseteq X$ with n objects, a query $\mathbf{q} \in X$, and a positive integer $k \leq n$, the NNS returns a set $R^* = \{\mathbf{o}_1^*, \mathbf{o}_2^*, \dots, \mathbf{o}_k^*\}$ of k objects from O with the top- k smallest distances to $\mathbf{q}$ on distance function δ .

In many applications, the number of objects involved in nearest neighbor search has grown significantly, making exact NNS computationally expensive. For example, exact NNS on a large number of high-dimensional Euclidean space vectors is impractical in many cases due to the curse of dimensionality. Hence, approximate nearest neighbor search (ANNS) is widely used, which sacrifices little query quality for higher query efficiency. In the literature, *recall* is commonly used to measure the quality of ANNS. If R^* is the exact NNS result and R is the ANNS result, the recall is defined as $\text{Recall}(R) = \frac{|R \cap R^*|}{|R^*|} = \frac{|R \cap R^*|}{k}$. The goal of ANNS query processing is to maximize query throughput while keeping a high recall.

In many applications, each object in the set O is associated with a specific attribute (e.g., date, price), and users may want to perform NNS on objects whose attributes fall within a given range. For example, if vectors represent image embeddings and the attribute is the publication date, users might seek images similar to a given image that were published within a specific date range. This problem is known as NNS with a range filter, formally defined as follows:

Definition 2.2 (NNS with Range Filter). Let (X, δ) be a metric space. Given an object set $O \subseteq X$ where each object $\mathbf{o} \in O$ has an attribute $\phi(\mathbf{o})$, a query $\mathbf{q} \in X$, a positive integer k , and a range $Q = [\ell, r]$, the NNS with range filter returns the nearest neighbors to $\mathbf{q}$ on δ within the set $O_Q = \{\mathbf{o} \in O \mid \mathbf{o} \in O, \phi(\mathbf{o}) \in Q\}$, i.e., the objects satisfying the range filter on attribute ϕ .

For NNS with range filter queries, we also focus on the approximate version to gain high query efficiency while achieving high recall. In this paper, we focus on high-dimensional **Euclidean spaces** with δ being the **Euclidean distance**, the most widely used setting in existing vector databases [33, 35, 39]. But we note that our method can be applied to other metric spaces as well.

Example 2.3. As shown in Fig. 2, consider a set of objects $O = \{\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_7\}$ where each object is associated with an attribute value on ϕ . For example, $\mathbf{o}_2$ has an attribute value of 1 on ϕ . Given a query vector $\mathbf{q}$, represented by a cross, performing a NNS on O returns $\mathbf{o}_1$, as it is the closest to $\mathbf{q}$. When we perform an NNS with a range filter on attribute ϕ , where the query range is $[3, 6]$ and k is set to 2, the subset after applying the range filter is $O_Q = \{\mathbf{o}_3, \mathbf{o}_4, \mathbf{o}_6, \mathbf{o}_7\}$. Then, $\mathbf{o}_4$ and $\mathbf{o}_6$ are the query answers that meet the filter condition.

2.2 NSW and HNSW

Given a set O of n vectors in d -dimensional space $\mathbb{R}^d$, graph-based methods construct a directed proximity graph G . Each vertex in G corresponds to a vector in O , and directed edges connect

Algorithm 1: GreedySearch

Input: Graph G , entry point $entry$, query vector $\mathbf{q}$, ef
Output: Query result

```

1 Mark  $entry$  as visited and set  $C \leftarrow \{entry\}$ ,  $W \leftarrow \{entry\}$ ;
2 while  $|C| > 0$  do
3    $u \leftarrow$  extract nearest element from  $C$  to  $\mathbf{q}$ ;
4    $f \leftarrow$  get furthest element from  $W$  to  $\mathbf{q}$ ;
5   if  $\delta(u, \mathbf{q}) > \delta(f, \mathbf{q})$  then break;
6   for  $v \in N_G(u)$  do
7     if  $v$  is not visited then
8       Mark  $v$  as visited;
9       if  $\delta(v, \mathbf{q}) < \delta(f, \mathbf{q})$  or  $|W| < ef$  then
10         $C \leftarrow C \cup \{v\}$ ,  $W \leftarrow W \cup \{v\}$ ;
11        if  $|W| > ef$  then
12          Remove  $f$  from  $W$ ;
13 return  $W$ ;
```

pairs of vertices based on their proximity. Let u and v be vertices representing vectors $\mathbf{o}_u$ and $\mathbf{o}_v$, respectively, with $\delta(u, v)$ denoting the distance between them. The (out-)neighbors of a vertex u , denoted $N_G(u)$, are selected based on certain proximity rules.

NSW. NSW (Navigable Small World) [22] is a proximity graph that supports updates. In NSW, the edge density between near vertices is higher, but it also retains some edges between vertices that are far apart to facilitate routing. The NSW graph is constructed by continuously inserting elements in a random order, connecting them bidirectionally to M nearest neighbors among previously inserted elements found through a greedy search.

The ANNS and build processes on NSW adopt the same greedy search algorithm, as shown in Alg. 1. For simplicity, we use $\delta(u, \mathbf{q})$ to denote the distance between $\mathbf{o}_u$ and the query vector $\mathbf{q}$. The ANNS begins at an initial entry vertex $entry$, returning a candidate set that contains the vectors closest to $\mathbf{q}$ among the visited vertices. To find the final set of k nearest neighbors, the algorithm selects the k closest vectors from the returned candidate set W . The parameter ef determines the size of W , thereby balancing the trade-off between search efficiency and accuracy: increasing ef broadens the search scope and improves accuracy, but at the cost of reduced efficiency.

NSW updates. Alg. 2 shows the pseudo-code for handling vertex insertion in NSW. Initially, the ef_{con} nearest vertices to u are retrieved using Alg. 1, and these vertices are selected as edge candidates. The candidates are then pruned to retain at most M edges, forming the neighborhood $N(u)$ (Lines 1–2). For each $v \in N(u)$, u is added to $N(v)$, and pruning is performed if $|N(v)| > M$ (Lines 3–6). An edge (u, v) is considered dominated if there exists an edge (u, w) in the graph such that $\delta(u, w) < \delta(u, v)$ and $\delta(v, w) < \delta(u, v)$. During pruning, candidates are sorted by their distance to u (Line 8), and edges not dominated are iteratively added to the set R (Lines 10–14). For vertex deletions, a vertex is simply marked as deleted, and the traversal will not include marked vertices into the answer.

HNSW. HNSW (Hierarchical Navigable Small World) is a popular graph-based method for ANNS. Unlike NSW that use a single proximity graph, HNSW leverages a multi-layer structure inspired by skip lists, where each layer contains a NSW graph. Assume that HNSW has L layers. The NSW at layer i is denoted as $H[i]$. The number of vertices decreases exponentially from the base layer $H[0]$ to the top layer $H[L]$, similar to the hierarchical structure of skip lists. Like NSW, HNSW is also built by inserting vertices incrementally into an initially empty graph.

To handle ANNS with HNSW, the search begins at the top layer and descends through the layers to reach layer 0, where the main search is performed. Starting from layer L , it finds the vertex closest to query $\mathbf{q}$ at each layer, using it as the entry point for the next layer, until reaching layer 0. Once at layer 0, a greedy search, $\text{GreedySearch}(H[0], ep_0, \mathbf{q}, ef)$, is applied to find the closest vertices to $\mathbf{q}$. Finally, it returns the k nearest objects from the search.

HNSW updates. When inserting a vertex u into HNSW, it first determines the highest layer L_u in which u will appear, where L_u is determined by a truncated geometric distribution. After determining the layers, u is inserted into the NSW graphs using *GraphInsertion* from layer L_u down to layer 0. The maximum out-degree is $2M$ in layer 0 and M in all other layers. When deleting a vertex, it uses the same marking method as in NSW.

2.3 Existing Solutions

Prefiltering. Prefiltering is a straightforward method. It pre-sorts all elements based on attribute ϕ . Given a query vector $\mathbf{q}$ and a range filter $[l, r]$, it first does a binary search on the sorted attribute list to find the start and end positions of elements satisfying the range filter, and then scans this subset, computes the distance between $\mathbf{q}$ and each element, and returns the closest k element.

Postfiltering. Postfiltering method builds an ANNS index on all data. For a query vector $\mathbf{q}$ and a range filter $[l, r]$ on attribute ϕ , it first retrieves the $K = k$ nearest neighbors using the ANNS index. The range filter is then applied to these elements to check if at least k of them satisfy the filter. If fewer than k elements meet the criteria, K is multiplied by a constant factor $m > 1$, and the ANNS query is repeated. It continues until the stopping condition is met, e.g., K is sufficiently large. The final result is the closest elements to $\mathbf{q}$ that meet the range filter, based on the most recent ANNS query.

ACORN. ACORN [29] is an index for general-filtered ANNS based on HNSW. A straightforward approach for general-filtered ANNS is to only access vertices that satisfy the filter on HNSW, ensuring that the search results meet the filter. However, when the selectivity is low, this method may fail to guarantee search quality due to too few edges on the graph satisfying the filter. To address this issue, ACORN has improved HNSW using heuristic methods. ACORN increases the number of edges in the graph. During the search process, not only the vertices satisfying the filter in 1-hop are added to the search queue, but also those satisfying the filter in 2-hop are added. This ensures that each vertex can expand a sufficient number of candidates to guarantee search quality. ACORN focuses more on ANNS with general predicate filters and does not perform well on range filters, which is demonstrated in our experiment.

SeRF. An idea to perform range-filtered ANNS is to build a dedicated ANNS index for each possible query range on attribute ϕ . SeRF [40] is based on this concept, utilizing HNSW as the index for each range on attribute ϕ . To reduce redundancy, SeRF compresses the index by minimizing duplicate edge storage. However, this method leads to the creation of $O(n^2)$ indices, which becomes impractical for large datasets. SeRF addresses this issue by recognizing that certain edges are shared across indices for continuous ranges. When an edge appears in the indices for intervals $[x, y]$, for all $x \in [l, r]$ and $y \in [b, e]$, SeRF avoids storing the edge multiple times. Instead, it stores the edge once, associating it with the four values l, r, b, e . During a query with a range filter $[L, R]$, SeRF performs ANNS on the HNSW index comprising vertices that satisfy the range filter and edges where $L \in [l, r]$ and $R \in [b, e]$. If the attribute values are independent of the object set, the expected space overhead of SeRF is $O(Mn \log n)$. However, in the worst case, space consumption can grow to $O(nM^2)$, and construction time may reach $O(M^2n^2)$. To mitigate this, SeRF employs a compression technique that omits the storage of certain edges. While this reduces space usage, excessive pruning can degrade the quality of query results. Besides, SeRF does not support arbitrary data insertion or deletion, making it unsuitable for large datasets that require dynamic updates.

Algorithm 2: GraphInsertion

Input: Graph G , node u , entry node $entry$, Parameters M, ef_{con}

```

1  $C \leftarrow GreedySearch(G, u, entry, ef_{con});$ 
2  $N_G(u) \leftarrow prune(C, u, M);$ 
3 for  $v \in N_G(u)$  do
4    $N_G(v) \leftarrow N_G(v) \cup u;$ 
5   if  $|N_G(v)| > M$  then
6      $N_G(v) \leftarrow prune(N_G(v), v, M);$ 
7 procedure  $prune(C, u, M):$ 
8   Sort elements in  $C$  by the distance to  $u$ ;
9    $R \leftarrow \emptyset;$ 
10  for  $v \in C$  do
11    if not exists  $w \in R$  s.t.  $\delta(v, w) < \delta(u, v)$  then
12       $R \leftarrow R \cup \{w\};$ 
13      if  $|R| = M$  then
14        break;
15  return  $R;$ 
```

SuperPostfiltering. SuperPostfiltering [12] avoids building indices for all $O(n^2)$ possible query ranges for attribute ϕ . It constructs graph indices for ranges on attribute ϕ within a predefined range set R . This set consists of $\log_\beta n$ levels, where β is a constant. At level i , the ranges are defined as $R[i] = \{[j \cdot \beta^i + 1, (j+2) \cdot \beta^i] \mid j \geq 0 \wedge (j+2) \cdot \beta^i \leq n\}$. For a query range containing m elements, a corresponding range in $R[i]$ can be found such that $\beta^{i-1} \leq m \leq \beta^i$, ensuring the number of elements in this range does not exceed $2\beta m$. Once the appropriate range is identified, Postfiltering is applied on the index for that range. While offering good query performance, significant overlap between ranges at each level leads to high space consumption. Despite the theoretical space complexity of $O(nM \log n)$, the actual space usage can be several times larger than the original dataset. Furthermore, SuperPostfiltering has the same issue as SeRF that does not support updates.

iRangeGraph. The iRangeGraph index [38] is built on a static segment tree over attribute ϕ , where each node corresponds to an HNSW index covering the elements within the range of that node. For query, iRangeGraph does not perform a greedy search on a preconstructed graph at a specific node in the segment tree. Instead, it constructs the search graph during the query process. For an ANNS query with a range filter $[l, r]$, when the greedy search accesses an element u , the algorithm dynamically searches for M out-neighbors v that satisfy $l \leq \phi(v) \leq r$. This is done by traversing the segment tree from the root to the leaf node associated with u , resulting in $O(\log n)$ nodes being examined. These neighbors are then added to the search queue. iRangeGraph achieves a good trade-off between efficiency and accuracy. Yet, since segment tree is a static structure, it cannot efficiently handle dynamic data.

3 Our solution

Next, we elaborate on our proposed DIGRA index. In Sec. 3.1, we introduce the dynamic multi-way tree structure, T , which organizes data points in order based on their attribute values ϕ . At each node u in T , we maintain an NSW graph index, constructed for all vectors falling within the sub-tree rooted at node u . This dynamic structure enables efficient handling of data insertions and deletions while keeping the index updated seamlessly. In Sec. 3.2, we explain the index construction process. Next, we explain how to process queries in Sec. 3.3; we show how to use the concept of α -approximate range coverage to efficiently identify at most two nodes in T that can include

Table 1. Frequently used notations

Notation	Description
O, n	The set O with n objects
$\phi(\mathbf{o})$	The attribute value of object $\mathbf{o}$
$\delta(u, v)$	The distance between u and v
$Q = [\ell, r], \mathbf{q}$	The query range and the query vector
O_Q	The set of objects whose attribute falling into Q
M	The maximum out-degree in the graph
ef, ef_{con}	The size of the candidate set in graph search and graph construction, respectively
T, B	The tree index T and its maximum degree B
$u.key, u.child$	The key and the child set of node u
$u.en, u.G$	The search entry and the NSW graph of node u
H_u	The implicit hierarchical graph structure on u
O_u	The set of objects falling into the subtree of u

all vectors satisfying the range filter. This reduces the need to scan irrelevant portions of the tree, thereby improving query performance. In Sec. 3.4, we introduce our *lazy update mechanism*, designed to maintain high update efficiency in the NSW graph indices during dynamic changes. We show that the amortized time complexity for each update can be bounded by $O(c_{upd} \cdot \log n)$, where c_{upd} is the cost of inserting an object into the NSW graph. This balances the need for efficient updates while preserving query performance and accuracy. Table 1 shows the frequently used notations in our solution.

3.1 Index Structure

The proposed DIGRA index uses a multi-way tree structure T to manage the entire object set. To distinguish, we use "node" in tree structures and "vertex" in graph structures. The tree structure T is similar to a B-tree, where each node can store up to B children, and it separates its children by storing several key values. For each object $\mathbf{o}$, we use attribute value $\phi(\mathbf{o})$ as the key. For a node u in T with t keys and $t + 1$ children, we use $u.child[i]$ to represent its i -th child, and $u.key[i]$ to represent its i -th stored key value. Thus, all attribute values of objects in the subtree of $u.child[i]$ are guaranteed to be within the range of $(-\infty, u.key[0])$ for $i = 0$; range $[u.key[i - 1], u.key[i])$ for $1 \leq i < t$; range $[u.key[t - 1], +\infty)$ for $i = t$. For node u , we maintain an NSW-style search graph, represented as $u.G$. It is constructed for set O_u , the object set containing all the objects stored in the subtree rooted at u . To speed up the search, we additionally maintain an entry node for node u , represented as $u.en$, which is a leaf node in the subtree of u to help obtain the search entry in the current graph $u.G$. Each node in T further stores a value *layer* to indicate its level, with leaf nodes having a layer value of 0. For a non-leaf node u , its *layer* value is the layer value of its child plus one, i.e., $u.layer = u.child[0].layer + 1$. Similar to B-trees, DIGRA guarantees that all child nodes of any given node are assigned identical layer values. An important property of the multi-way tree structure in DIGRA is that we keep the tree T weight-balanced, whose definition is as follows.

Definition 3.1. (Weight Balance) Let T be a multi-way tree with a branching factor B . The tree T is said to be *weight balanced* if: For every non-root node u in T at layer i , the size of u , denoted by $size(u)$, satisfies $\frac{B^i}{4} \leq size(u) \leq B^i$, where $size(u)$ is defined as the number of leaf nodes in the subtree rooted at u .

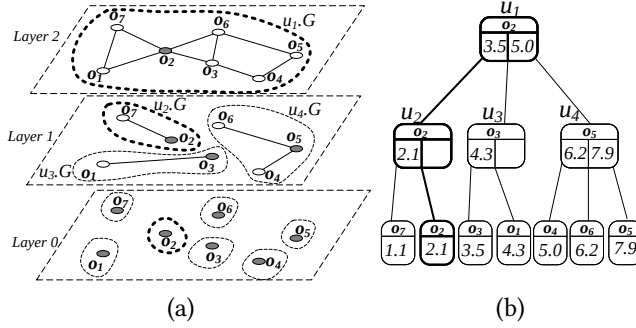


Fig. 3. The DIGRA index for object set O in Figure 2.

The weight of a node u is defined as its $size(u)$. We will use this condition to develop a lazy, weight-based update strategy that ensures update costs remain bounded. For now, we assume that the index is weight-balanced in our discussion of Sec.s 3.1-3.3.

LEMMA 3.2. *The height of T is $O(\log n)$.*

It is straightforward to verify using the weight balance condition and thus omitted. As to be detailed in Sec. 3.3, for any arbitrary range $Q = [\ell, r]$ on the attribute ϕ , there exist at most two nodes u and v in the tree T such that the union of their associated object sets, $O_u \cup O_v$, includes all objects O_Q that satisfy the range Q . Formally, this relationship is expressed as: $O_Q \subseteq O_u \cup O_v$. It is important to note that this approach may result in the inclusion of objects outside the specified range Q . Yet, we will demonstrate that the number of such irrelevant objects is bounded by a constant factor, ensuring that the majority of the objects within $O_u \cup O_v$ are relevant. Next, we utilize the NSW indices maintained at nodes $u.G$ and $v.G$ to incrementally retrieve the ANNS results, thereby preserving the efficiency of the ANNS and avoid retrieving many irrelevant objects whose attribute value on ϕ is out of the range Q .

One of our key observations is that our multi-way tree inherently includes a hierarchical structure. Consequently, we maintain an NSW index at each node without introducing an additional hierarchical layer. During the search algorithm, we enable skip-based searching, allowing an ANNS search within a node to leverage the graph indices of its subtree nodes to locate the appropriate entry points. From this perspective, the proposed DIGRA index resembles a specialized HNSW structure. Specifically, starting from the root node, it corresponds to a comprehensive NSW graph akin to the bottom layer in HNSW. As we traverse down the tree layers, this graph becomes partitioned according to the objects stored in the child nodes.

Therefore, we can implicitly maintain a hierarchical graph structure H_u similar to HNSW on u , which can help us efficiently perform ANNS. H_u includes all the graphs of the nodes on the path from the leaf node where $u.en$ is located to u , with $u.G$ as the bottom layer of H_u . This design offers several advantages. First, each layer retains only a single NSW graph, ensuring efficient updates during our lazy weight-based updates. Second, during the search, the ability to utilize child nodes to identify entry vertices helps maintain query efficiency comparable to that of HNSW.

Example 3.3. Consider again the set O in Fig. 2. We construct a DIGRA index for this dataset with a branching factor $B = 3$. The resulting balanced three-layer tree T , maintained by the DIGRA index, is illustrated in Fig. 3(b). For each node u in T , an NSW index is built for the corresponding subset O_u . For example, for node u_1 , O_{u_1} includes all objects in O , so the graph index maintained at u_1 spans all objects, as shown in Layer 2 of Fig. 3(a). Similarly, for node u_4 , O_{u_4} contains the objects o_4, o_5, o_6 . Thus, the NSW index $u_4.G$, illustrated in Layer 1 of Fig. 3(a), is constructed over these

Algorithm 3: DIGRA-Build**Input:** The sorted objects O , construction parameter M, ef_c **Output:** Root node of index DIGRA

```

1 Initialize queues  $Q, h \leftarrow 0$ ;
2 for  $o$  in  $O$  do
3    $nd \leftarrow createNode()$ ,  $nd.en \leftarrow o$ ;
4    $Q_0.push(nd)$ 
5 while  $|Q_h| > 1$ : do
6   while  $Q_h$  is not empty do
7     if  $|Q_h| \leq B$  then  $branch \leftarrow |Q_h|$ ;
8     else  $branch \leftarrow \lceil B/2 \rceil$ ;
9      $u \leftarrow createNode()$ ;
10     $u.child \leftarrow$   $branch$  nodes dequeued from  $Q_h$ ;
11     $Refresh(u, M, ef_c)$ ;
12     $Q_{h+1}.push(u)$ ;
13   $h \leftarrow h + 1$ ;
14 return  $Q_h.front()$ ;

```

three objects. For graph indices at layers other than 0, the entry point of each graph is highlighted in gray in Fig. 3(a) and maintained at the corresponding node in Fig. 3(b). For instance, o_5 is designated as the entry point for the graph $u_4.G$. The NSW along the path from a node u to the leaf where its entry is located can form a hierarchical graph structure denoted as H_u . For example, the graph $u_1.G$, $u_2.G$, o_2 in Fig. 3(a) corresponding to the path u_1, u_2, o_2 in Fig. 3(b) can be regarded as a hierarchical structure, which is denoted as H_{u_1} .

Comparing B+-tree and DIGRA. Although both B+-tree and DIGRA are limited-degree multi-way tree structures where all elements reside in the leaf nodes, DIGRA index employs distinct designs for ANNS with range filtering, in contrast to the traditional B+-tree. First, DIGRA does not require scan operations, so leaf nodes do not include linked-list pointers. Second, in addition to keeping a pointer to NSW in each node, we also store an entry which is a leaf node in the subtree to guide the implicit construction of HNSW using the tree structure. Third, while our update is based on splitting and merging, we maintain an entire NSW graph for the set of objects O_u in the subtree rooted at u for each u in T . Hence, we cannot rely on a B+-tree-only update method; instead, updates must address both the graph and tree structures. When nodes split or merge, the objects within their subtrees change, requiring corresponding NSW updates. To handle this efficiently, we have designed an update procedure that uses a lazy update strategy to ensure manageable time complexity.

3.2 Index Construction

The key idea of the DIGRA index construction is to build a dynamic multi-way tree from the bottom up, where each node maintains an NSW graph covering the objects in its subtree. We start by placing data objects into leaf nodes and constructing an NSW graph at each leaf. These leaf nodes are then grouped into higher-level nodes, merging the prebuilt graphs of their children to avoid rebuilding from scratch. This process continues iteratively until the entire tree is formed. Next, we introduce the details of index construction.

Alg. 3 shows the pseudo-code for building the DIGRA index. We first create leaf nodes for all objects and push them into Q_0 (Lines 2-4). From bottom to top, we iteratively extract *branch* nodes from the queue of the lower layer as children of newly created node, construct a graph on the new

Algorithm 4: Refresh

Input: node u , construction parameter M, ef_{con}

```

1  $u.en \leftarrow selectEn(u), u.layer \leftarrow u.child[0].layer + 1;$ 
2 for  $o \in O_u$  do
3    $R \leftarrow \emptyset;$ 
4   for  $v \in u.child$  do
5     if  $u \in O_v$  then  $R \leftarrow R \cup N_{v,G}(o);$ 
6     else
7        $entry \leftarrow findEntry(H_v, o);$ 
8        $C \leftarrow GreedySearch(s.G, entry, o, ef_{con});$ 
9        $R \leftarrow R \cup prune(C, o, M);$ 
10   $N_{u,Graph}(o) \leftarrow prune(R, o, M);$ 

```

node, and push it into the queue of the new layer (Lines 5-13). The *branch* is usually set to be $\lceil B/2 \rceil$ here. This process repeats until only one node remains in the new layer. We return this node as the root (Line 14).

In Alg. 4, we show the steps of building the NSW index for node u . First, we set entry and layer for u (Line 1). As we can obtain a multi-layer structure by visiting any path from a leaf node to u , we randomly select a leaf node from the subtree of u as its entry en . Then, we need to build the NSW for the new node by identifying edges for each o in the subtree of u (Lines 2-10). Recalling the process of building NSW, it first finds the closest ef_{con} vertices in the graph for each vertex. If all ef_{con} vertices are selected as neighbors of this vertex, the graph density will be too high. Hence, through *prune* in Alg. 2, only M vertices are chosen as neighbors. The intuition behind pruning is that if two vertices are close to each other, there is likely to be an edge between them, so we only keep one of them to be a neighbor to sparsify the graph and ensure search performance. Yet, since the NSW is built incrementally starting from an empty graph, vertices inserted earlier will have difficulty finding vertices inserted later as neighbors. To overcome this, for each o , we first independently find ef_{con} neighbors and select M edges through pruning in the graph of each child of u except the child that o belongs to (Lines 5-10). Then we further sparsify the found $M \cdot |u.child|$ edges to M edges through pruning (Line 10). This allows each vertex to obtain information from the entire set O_u , ensuring search performance. Note that since H_o is an implicit hierarchical NSW structure, we use $findEntry(H_u, o)$ to search H_v from the top to bottom to find the entry closer to o , which is the same as HNSW, thereby improving the speed (Line 7). Next, we show an example for the index construction.

Example 3.4. Still consider the set O of objects in Fig. 2. To build our DIGRA index with $B = 3$ (see Fig. 3), we first sort the objects o_1 through o_7 by their attribute values on ϕ . These objects form the leaf nodes at Layer 0. We then group the Layer-0 nodes into sets of size $\lceil B/2 \rceil = 2$, placing any remaining nodes (no more than B) into a single set. From this procedure, we obtain the following groups: $\{o_7, o_2\}, \{o_3, o_1\}$, and $\{o_4, o_6, o_5\}$. Next, we create nodes u_2, u_3 , and u_4 at Layer 1, each having one of these groups as its children. At this point, we also construct an NSW graph within each node. Since Layer 1 contains only three nodes (no more than B), we create a single node u_1 at Layer 2 whose children are u_2, u_3 , and u_4 . To build u_1 's NSW graph (denoted $u_1.G$), we apply the *Refresh* process. Specifically, each object in O_{u_1} (the union of all objects in u_2, u_3 , and u_4) locates neighbor candidates in $u_2.G, u_3.G$, and $u_4.G$. For example, object o_6 finds o_5 as a neighbor candidate in $u_4.G$. Using *GreedySearch*, it then locates o_2 from $u_2.G$ and o_3 from $u_3.G$. After a pruning step (which retains all vertices in this example), $\{o_5, o_2, o_3\}$ is the final neighbor set of o_6 in $u_1.G$.

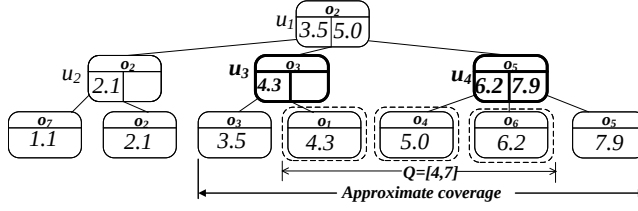


Fig. 4. Example of DIGRA query

Lemma 3.5 shows the time complexity of the *Refresh* operation. With Lemma 3.5, we further obtain the time complexity of building DIGRA index and the space occupied by DIGRA in Theorem 3.6. All omitted proofs can be found in our technical report [1].

LEMMA 3.5. *For node u in the DIGRA index, the time cost of performing the refresh operation is $O(\text{size}(u) \cdot c_{upd})$.*

THEOREM 3.6. *Algorithm 3 constructs a DIGRA index on O with $O(Mn \log n)$ space complexity in $O(n \log n \cdot c_{upd})$ time.*

3.3 Query Processing

Given a range filter Q , the main idea of DIGRA for query processing is to identify a subset of nodes whose subtrees collectively contain O_Q , the set of objects satisfying the filter. We then run ANNS on the NSW index at each chosen node, merge the partial results, and return the top- k answers. As we will show, our approach can locate $O(B \log n)$ such nodes, ensuring that only objects in $[\ell, r]$ are included. However, querying all $O(B \log n)$ nodes still incurs relatively high costs. To reduce these costs, we propose an alternative solution requiring at most two nodes that jointly cover O_Q . While this introduces a bounded number of out-of-range objects, it significantly improves query efficiency compared to the disjoint exact coverage solution requiring $O(B \log n)$ searches.

We begin by presenting the solution that identifies $O(B \log n)$ nodes to obtain a *disjoint exact coverage* of O_Q , defined below.

Definition 3.7 (Disjoint Exact Coverage). A set of nodes $C = \{u_1, u_2, \dots, u_x\}$ in tree T *disjointly covers* an arbitrary range $Q = [\ell, r]$ exactly if: $\bigcup_{u \in C} O_u = O_Q$ and $O_{u_i} \cap O_{u_j} = \emptyset$ for all $i \neq j$, where O_Q is the set of objects with attribute ϕ falling into in Q , and O_u is the set of objects in the subtree rooted at u .

LEMMA 3.8. *DIGRA can identify a set C of $O(B \log n)$ nodes in T that disjointly and exactly covers an arbitrary query range $Q = [\ell, r]$ on the attribute ϕ in $O(B \log n)$ time.*

Using Lemma 3.8, we can perform an ANNS on the NSW graph $u.G$ for each node $u \in C$, merge the results and return the top- k answers. Yet, this still incurs high query costs due to $O(B \log n)$ ANN searches. To alleviate this issue, we find that it is feasible to include some objects with attribute value on ϕ out of the range $[\ell, r]$ and then prune them to save costs. The key is to bound such irrelevant objects. Thus, we propose the α -approximate range coverage.

Definition 3.9. (α -Approximate Range Coverage) For a range $R = [\ell, r]$ over dataset O , let O_R denote the objects in O falling within R . If a range $R' = [\ell', r']$ with $R \subseteq R'$ satisfies $\frac{|O_{R'}|}{|O_R|} \leq \alpha$ for some constant α , R' is an α -approximate coverage of R .

THEOREM 3.10. *For any range $Q = [\ell, r]$, we can find up to two disjoint nodes u and v in T within $O(\log n)$ time whose range $Q' = [\ell', r']$ is a $4B$ -approximate coverage of Q , i.e., $|O_u \cup O_v|/|O_Q| \leq 4B$.*

Algorithm 5: DIGRA-Query

Input: Query vector $\mathbf{q}$, range $[\ell, r]$, query parameters k, ef
Output: Query result

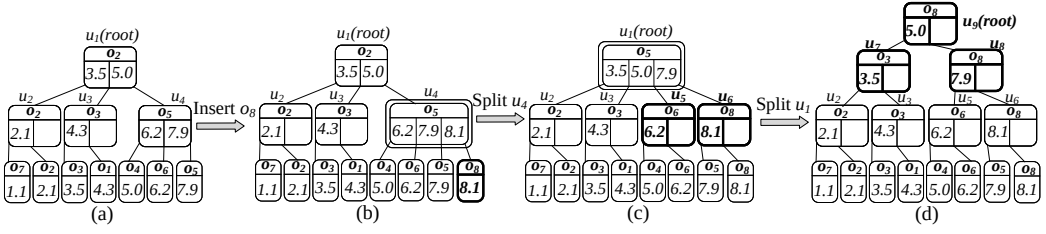
```

1  $hNode \leftarrow findHighNode(root, \ell, r)$ ;
2 if  $hNode.layer == 0$  then return  $\{hNode.en\}$  ;
3  $lId \leftarrow findPostion(hNode, \ell)$ ;
4  $rId \leftarrow findPosition(hNode, r)$ ;
5  $entryList \leftarrow emptylist$ ;
6 if  $rId > lId + 1$  then
7    $entry \leftarrow findEntry(H_{hNode}, \mathbf{q})$ ;
8    $entryList.push(entry)$ ;
9    $G \leftarrow hNode.G \cup (\bigcup_{ch \in hNode.child} ch.G)$ ;
10 else
11    $lNode \leftarrow findNode(hNode.child[lId], \ell)$ ;
12    $rNode \leftarrow findNode(hNode.child[rId], r)$ ;
13    $lEntry \leftarrow findEntry(H_{lNode}, \mathbf{q})$ ;
14    $rEntry \leftarrow findEntry(H_{rNode}, \mathbf{q})$ ;
15    $entryList.push(lEntry, rEntry)$ ;
16    $lG \leftarrow lNode.G \cup (\bigcup_{ch \in lNode.child} ch.G)$ ;
17    $rG \leftarrow rNode.G \cup (\bigcup_{ch \in rNode.child} ch.G)$ ;
18    $G \leftarrow lG \cup rG \cup hNode.G$ ;
19  $W \leftarrow RangeGreedySearch(G, entryList, \mathbf{q}, ef, \ell, r)$ ;
20 return  $k$  objects closest to  $\mathbf{q}$  in  $W$ ;

```

Theorem 3.10 enables us to focus on at most two nodes, u and v , and their associated NSW indices, $u.G$ and $v.G$, to efficiently process the search. Our results demonstrate that this strategy yields approximately a 4x speed-up in query processing compared to directly solving the problem using disjoint exact coverage. The detailed experimental results are available in our technical report [1]. Building on this idea, we now present the detailed query algorithm.

Query algorithm. Alg. 5 shows the pseudo-code for answering a query. First, we find the lowest layer node $hNode$ that covers the range $[\ell, r]$ in the tree in $O(\log n)$ time (Line 1). Then: (i) If $hNode.layer = 0$, the range covers a single leaf, so we directly return that leaf's entry (Line 2). Next, we find lId and rId , the ids of the children which contain ℓ and r using the *findPosition* function (Lines 3-4), and check the relationship between lId and rId to determine if $hNode$ can serve as an approximate range coverage (Line 6). (ii) If range $[\ell, r]$ fully covers at least one child of $hNode$, i.e. $rId > lId + 1$, we perform ANNS on $hNode$ (Lines 6-9). (iii) Otherwise, we search with two nodes (Lines 11-20). Specifically, we use the function *findNode* (Lines 11-12) to locate two nodes, each having at least one child covered by $[\ell, r]$. From the lId/rId child of $hNode$, *findNode* recursively visits the right/left-most children until they are fully covered by $[\ell, r]$, taking $O(\log n)$ time by the tree's height. In addition, we optimize NSW search by leveraging a hierarchical structure. First, we use the hierarchy to find entries closer to $\mathbf{q}$, which is described in Sec. 3.2, to accelerate search convergence (Lines 7, 12-13). Second, to enhance the search quality, we expand edge coverage by incorporating edges from the same vertices at the graphs on the children (Lines 9, 16-17) and the ancestor (Line 18) of the nodes to be searched. Since there can be two search entries, and not all vertices in the graph satisfy the range filter, we use the *RangeGreedySearch* algorithm, similar to Alg. 1, but with two modifications: (i) initialization uses multiple entries, and (ii) a vertex v is added to the set W only if $\ell \leq \phi(v) \leq r$. Theorem 3.11 shows this yields a $4B$ -approximate coverage.

Fig. 5. An example of the insertion with DIGRA ($B = 3$).

THEOREM 3.11. *Given range Q and dataset O , Alg. 5 returns the ANNS results on a set O' such that $O_Q \subseteq O'$ and $|O'|/|O_Q| \leq 4B$.*

Example 3.12. Consider the DIGRA index in Fig. 3 and let the query range Q be $[4, 7]$. The exact coverage for this range is $O_Q = \{o_1, o_4, o_6\}$, highlighted by gray boxes in Fig. 4. Besides, we further find that the values 4 and 7 lie in different children of u_1 , namely u_3 and u_4 and each node contains at least one child whose attribute lies within Q . Hence, we choose u_3 and u_4 (shown in bold) as an $4B$ -approximate coverage for Q . This coverage may include objects outside of Q (e.g., o_3 and o_5), but the total number of such irrelevant objects is bounded by the α -approximate coverage guarantee. Finally, we run *RangeGreedySearch* on $u_3.G$ and $u_4.G$. For query processing, objects with attributes lie outside of Q are filtered out.

3.4 Index Update

The key to updating the DIGRA index is to ensure that while maintaining the tree's balance condition, the ANN index maintained by each node remains up-to-date and consistent with the updated set in the corresponding subtree.

Our update algorithms preserve the tree's balance and the ANN index's consistency, both crucial for efficient and accurate queries. They operate in two steps: (i) descend from the root to locate the node needing an update, refreshing each affected node's ANN index along the way; and (ii) backtrack to detect balance violations, rebalancing via split or merge when necessary. The first step costs no more than updating $O(\log n)$ vanilla NSW graphs, since a balanced tree's search path is $O(\log n)$. Hence, efficient updates require bounding the second step's overhead. Splitting or merging a node u of size $\text{size}(u)$ takes $O(\text{size}(u) \cdot c_{\text{upd}})$. Frequent rebalancing can be costly when $\text{size}(u)$ reaches $O(n)$.

To address this, we employ a *lazy update mechanism* that permits each initially balanced node u to tolerate $\Omega(\text{size}(u))$ updates before triggering a rebalance. This strategy bounds the amortized overhead for each update to $O(c_{\text{upd}} \log n)$ time, significantly reducing the update cost. Next, we show the details of the update algorithms.

Insertion of DIGRA index. Alg. 6 outlines the pseudo-code of insertion. If the current tree is empty, we create a new node and make it the root node (Lines 1-2). Otherwise, starting from the root, we insert a new object o recursively (Line 4). We first find the position in tree to insert the new node, create a leaf node and insert this leaf node into the corresponding position (Lines 11-14). Then we insert o into the graph of each node on the backtracking path (Lines 15-26).

Inserting o into a graph follows the standard NSW insertion process, using the *GraphInsertion* algorithm described in Alg. 2.

Next, if any child of the node has greater than B children after the insertion, we split the child (Line 17). The splitting process is similar to B-tree splitting. When we split the node $n1$, we divide the key and children of $n1$ into two parts, keeping one part for itself and moving the other part becoming the new node $n2$, and insert $n2$ after $n1$ (Lines 18-21). As the subtree of $n1$ and $n2$ changed,

Algorithm 6: DIGRA-Insert

Input: object $\mathbf{o}$, construction parameter M, ef_{con}

```

1 if  $root = NULL$  then
2    $root \leftarrow createNode(), root.layer \leftarrow 0, root.en \leftarrow \mathbf{o}$ 
3 else
4    $insertNode(root, \mathbf{o});$ 
5   if  $|root.child| > B$  then
6      $nRoot \leftarrow createNode();$ 
7      $nRoot.child[0] \leftarrow root, root \leftarrow nRoot;$ 
8      $splitNode(root, 0);$ 
9 procedure  $insertNode(node, \mathbf{o}, M, ef_{con})$ :
10    $id \leftarrow findPosition(node, \mathbf{o});$ 
11   if  $node.layer = 1$  then
12      $newNode \leftarrow createNode();$ 
13      $newNode.en \leftarrow \mathbf{o}, entry \leftarrow \mathbf{o};$ 
14      $insertKeyAndChild(node, id, \mathbf{o}, newNode);$ 
15    $entry \leftarrow \text{vertex closest to } \mathbf{o} \text{ in } N_{node.child[id].G}(\mathbf{o});$ 
16    $GraphInsertion(node.G, \mathbf{o}, entry, M, ef_{con});$ 
17   if  $|node.child[id].child| > B$  then  $splitNode(node, id, M, ef_{con});$ 
18 procedure  $splitNode(node, id, M, ef_{con})$ :
19    $n1 \leftarrow node.child[id], n2 \leftarrow createNode();$ 
20    $n1, n2 \leftarrow splitEvenly(n1);$ 
21    $insertChild(node, id, n2);$ 
22    $Refresh(n1, M, ef_{con}), Refresh(n2, M, ef_{con});$ 

```

we rebuild $n1.G$ and $n2.G$ by *Refresh* (Line 22). When the insertion finishes, if the number of children of the root exceeds B , we split the root and make it a child of the new root node (Lines 5-8).

We first analyze the time complexity of the *Split* operation:

LEMMA 3.13. *The time of a split operation to u is $O(\text{size}(u) \cdot c_{upd})$.*

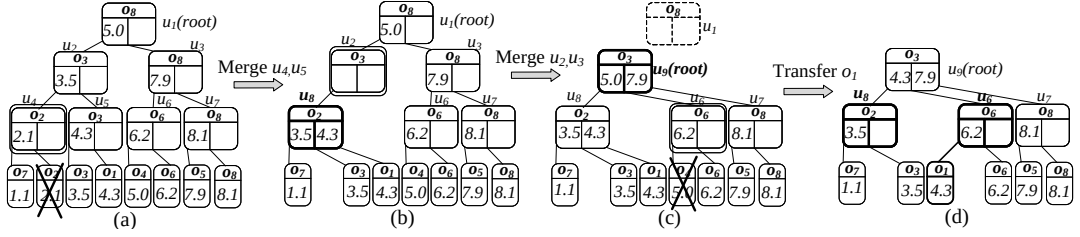
Assuming that after the last split on node u , the next split occurs after $\Omega(\text{size}(u))$ insertions in u 's subtree by using the lazy update strategy elaborated later, we have the following theorem:

THEOREM 3.14. *The DIGRA index can handle each insertion in $O(c_{upd} \log n)$ amortized time.*

Example 3.15. Continuing from Fig. 2, assume we have built the DIGRA tree in Fig. 5(a) with $B = 3$ using $\mathbf{o}_1$ through $\mathbf{o}_7$. We now wish to insert $\mathbf{o}_8$, which has $\phi(\mathbf{o}_8) = 8.1$. First, we locate $\mathbf{o}_8$'s proper position in the tree and create a new leaf node (shown in bold in Fig. 5(b)). This insertion causes u_4 's number of children to exceed 3, prompting us to split u_4 into two new nodes, u_5 and u_6 . We rebuild their graphs, $u_5.G$ and $u_6.G$, using *Refresh*, and connect both nodes to their parent, u_1 , as seen in Fig. 5(c). Afterwards, u_1 also has more than 3 children and, being the root, must be split as well. We create a new root u_9 . The split of u_1 produces two new children, u_7 and u_8 . We then rebuild $u_7.G$, $u_8.G$, and $u_9.G$ using *Refresh*. This process yields the final tree, shown in Fig. 5(d).

Deletion of DIGRA index. Alg. 7 outlines the pseudo-code for dealing with deletions.

We first apply the same marking method as in NSW. We mark the object as deleted (Line 1). When doing *RangeGreedySearch* on any graph in query processing, we will put vertices marked for deletion into the search queue C , but not into the result set W . This completes the deletion of $\mathbf{o}$ in all NSW graphs. Next, we will focus on the change in tree structures. We proceed with the

Fig. 6. An example of the deletion with DIGRA ($B = 3$).

recursive deletion process starting from the root (Line 2). We find the corresponding leaf node and delete it (Lines 5-6). After deleting the leaf node, we need to check if the number of children of $child[id]$ of the backtracked node is less than $\lceil B/2 \rceil$. If so, we look for a sibling node whose number of children is greater than $\lceil B/2 \rceil$. If we find such a node, we transfer a child from the sibling node and refresh the graphs of the two nodes (Lines 9-17). If we cannot find such a sibling node, we merge it with one of its siblings on the left or right (Lines 18-19). When merging $n1$ and $n2$, we append the children and keys of $n2$ to $n1$ (Lines 21-22). Then we remove $n2$, and rebuild the graph of $n1$ (Line 24). Finally, if the root node has only one child left after deletion, we will remove the root node and set its child as the new root node (Line 3). Similar to insertion, we present the time complexity of *Merge* and *Deletion* below.

LEMMA 3.16. *The time cost of merge operation is $O(\text{size}(u) \cdot c_{upd})$.*

Assuming that after the last merge on node u , the next merge on u occurs after $\Omega(\text{size}(u))$ deletions in its subtree by using the lazy update strategy explained later, we have the following theorem:

THEOREM 3.17. *The deletion of DIGRA index can handle each deletion operation in $O(c_{upd} \log n)$ amortized time.*

Example 3.18. Consider the object set from Example 3.15. We have constructed the tree structure of DIGRA shown in Fig. 6 (a) with $B = 3$ using o_1 to o_8 , and now we want to delete o_2 and o_4 sequentially. We locate the corresponding position for o_2 and delete the leaf node. After deletion, the number of children of u_4 is less than 2. We merge u_4 and u_5 into u_8 , and build $u_8.G$ by *Refresh*. In the structure shown in Fig. 6 (b), we find that u_2 has only one child, so we merge u_2 and u_3 into u_9 . Then we build $u_9.G$ by *Refresh* and set u_9 as a new root since u_1 has only one child. Then we delete o_4 from the index shown in Fig. 6 (c). As u_8 has 3 children, we transfer a child from u_8 to u_6 . Then we update $u_8.G$ and $u_6.G$ by *Refresh* and obtain the index shown in Fig. 6 (d).

Lazy update. The B-tree only limits the number of branches and is not a very effective balancing strategy in extreme cases. Consider a B-tree with $B = 3$. Suppose a node u has two children v_1 and v_2 , where all nodes in the subtree of child v_1 have 2 branches, and all nodes in the subtree of child v_2 have 3 branches. Thus we no longer have $\text{size}(u) = O(\text{size}(v_1))$, since $\text{size}(v_1) = 2^i$ and $\text{size}(u) = 3^i + 2^i$. This makes it difficult to obtain an α -approximate range coverage. To improve this, we trigger a split or merge when the balance condition introduced in Definition. 3.1 is not met. We replace the condition at line 17 in Alg. 6 with the check if the size of the node is greater than B^i , and replace the condition at lines 9,10,14 in Alg. 7 with the check if the size to the node is less or greater than $B^i/4$, where i is the layer of the node. To balance the tree, we do not need to perform split and merge on all of the nodes every update. According to the previous work [5], if we use such a balancing strategy, we have the following theorem:

THEOREM 3.19. *It takes $\Omega(\text{size}(u))$ insertions or deletions to make the balanced node u after a merge or split operation unbalanced again.*

Algorithm 7: DIGRA-Delete

Input: object $\mathbf{o}$, construction parameter M, ef_{con}

```

1  markAsDeleted( $\mathbf{o}$ );
2  eraseNode( $\text{root}, \mathbf{o}$ );
3  if  $|\text{root.key}| = 0$  then  $\text{root} \leftarrow \text{root.child}[0]$ ;
4  procedure eraseNode( $\text{node}, \mathbf{o}$ ):
5       $\text{id} \leftarrow \text{findPosition}(\text{node}, \mathbf{o})$ ;
6      if  $\text{node.layer} = 1$  then removeKeyAndChild( $\text{node}, \text{id}$ ) ;
7      else
8          eraseNode( $\text{node.child}[\text{id}], \mathbf{o}$ );
9          if  $|\text{node.child}[\text{id}].\text{child}| < \lceil B/2 \rceil$  then
10              if  $\text{id} > 0$  and  $|\text{node.child}[\text{id} - 1].\text{child}| > \lceil B/2 \rceil$  then
11                  redistributeKeys( $\text{node}, \text{id}$ );
12                  Refresh( $\text{node.child}[\text{id} - 1], M, ef_{con}$ );
13                  Refresh( $\text{node.child}[\text{id}], M, ef_{con}$ );
14              else if  $\text{id} < |\text{node.child}|$  and  $|\text{node.child}[\text{id} + 1].\text{child}| > \lceil B/2 \rceil$  then
15                  redistributeKeys( $\text{node}, \text{id}$ );
16                  Refresh( $\text{node.child}[\text{id}], M, ef_{con}$ );
17                  Refresh( $\text{node.child}[\text{id} + 1], M, ef_{con}$ );
18              else if  $\text{id} > 0$  then mergeNodes( $\text{node}, \text{id} - 1$ ) ;
19              else mergeNodes( $\text{node}, \text{id}, M, ef_{con}$ ) ;
20  procedure mergeNodes( $\text{node}, \text{id}, M, ef_{con}$ ):
21       $n1 \leftarrow \text{node.child}[\text{id}], n2 \leftarrow \text{node.child}[\text{id} + 1]$ ;
22       $n1 \leftarrow \text{append}(n1, n2)$ 
23      removeChild( $\text{node}, n2$ ),
24      Refresh( $n1, M, ef_{con}$ );

```

Recap that Theorems 3.14 and 3.17 both require this lazy update strategy to achieve the desired time complexity. The above theorem affirms that such a lazy update strategy is doable, showing the final time complexity of insertion and deletion to be both $O(c_{upd} \log n)$.

4 Future work

Next, we discuss possible extensions of our method, which are considered as our future work.

Memory limited method. Our method consists of tree structure and graph index. For the graph index that cannot fit entirely into memory, one way is to store the index on disk. Some disk-based graph indexes exist for ANNS, such as Diskann [14]. These graph indexes mostly optimize the layout on disk to improve locality and also cache some hot data in memory to reduce I/O costs. Besides, the multi-way tree is a disk-friendly structure. Thus, we can transform the graph index from NSW to disk-based graph index to implement DIGRA on disk. Another way is to partition a dataset into multiple clusters and store them on disk and retrieve the needed partitions into memory when necessary for query processing.

Concurrent updates. Although DIGRA is designed for single-threaded updates, extending it to a multi-threaded environment could serve as a future research direction. The NSW of each node in DIGRA is independent of the others. Additionally, if the tree structure remains unchanged, the NSW at one node will not be affected by changes at other nodes. Hence, we can treat the NSW as a whole and simply lock the node when updating it. When there are modifications to the tree structure, we may adopt a similar locking method as in B+-tree. However, the advanced B+-tree applies write locks only for structure modification, while DIGRA requires locking when updating

Table 2. Summary of datasets.

Dataset	Vector Type	Dim.	Attribute Type
SIFT	Image feature vectors	128	Synthetic
Redcaps	CLIP embeddings	512	Timestamp
GIST	Image feature vectors	960	Synthetic
WIT	ResNet-50 embedding	2048	Image size

the NSW. This presents challenges for implementing an efficient locking strategy. Additionally, when processing queries synchronously during updates, DIGRA needs to ensure the consistency of the NSWs stored in internal nodes, which differs from B+-trees that only consider consistency among the elements stored in leaf nodes.

5 Related Work

Approximate Nearest Neighbor Search. Approximate nearest neighbor search (ANNS) methods are generally categorized into three types: LSH-based [2, 3, 9, 17, 36], PQ-based [4, 6, 7, 18, 19, 38], and graph-based [11, 13, 23, 24, 32]. Locality-Sensitive Hashing (LSH) [17] hashes similar items into the same bucket with high probability. While LSH and its variants offer adjustable performance and theoretical error guarantees, they require more space to maintain accuracy compared to other methods. Product Quantization (PQ) [18] partitions high-dimensional space into multiple low-dimensional subspaces and quantizes each separately. PQ and its extensions [4, 6, 7, 19, 38] trade some accuracy for reduced space usage, making them suitable for billion-scale datasets. Graph-based methods [11, 13, 23, 24, 32] build a proximity graph where nodes represent vectors and edges connect similar vectors, using greedy search for ANNS. Notable graph-based approaches like HNSW [24], NSG [13], and DiskANN [32] achieve a strong balance between efficiency and accuracy and are widely adopted in industry.

ANNS with attribute filtering. Several algorithms and systems have been developed for attribute-filtered ANN queries [12, 14, 25, 33, 38–40]. Some systems, like Milvus [33], support general attribute filters and predicates, employing different strategies for various ANN searches. RII [25], based on PQ, first filters vectors by predicates before applying a query strategy. However, previous works [12, 38, 40] show that general methods are sub-optimal for specific attribute queries, where specialized designs often perform better. Beyond range-filtering [12, 38, 40], Filtered-DiskANN [14] target queries with attributes such as date, price range, or language.

6 Experiments

We compare the proposed DIGRA against states of the art in various aspects through experiments. Our method is a standalone code implemented using C++, without introducing any external packages. We have enabled all optimizations in the code of the competitors for experimental comparison. Our method, the same as other competitors, is an independent component based on the ANN index, and will not interfere with other system functionalities. All experiments are conducted on a Linux machine with an Intel Xeon(R) CPU at 2.30GHz and 768GB of memory using a single thread.

6.1 Experimental Settings

Datasets. We use the following four real-world datasets and their query settings tested in related research for range-filtered ANNS [12, 24, 40]: (i) SIFT, (ii) Redcaps, (iii) GIST and (iv) WIT. The attributes of SIFT and GIST are random integers from the range $[1, 10^4]$, while the attributes of Redcaps and WIT are real properties. The datasets are summarized in Table 2. For each dataset,

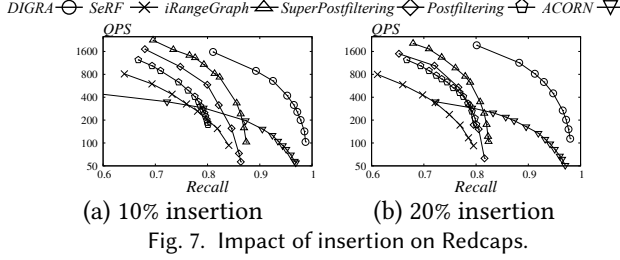


Fig. 7. Impact of insertion on Redcaps.

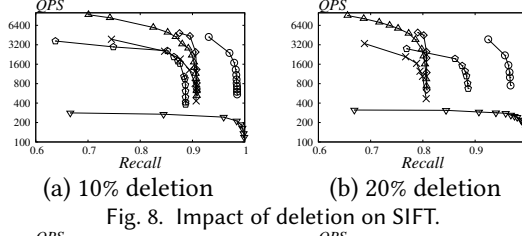


Fig. 8. Impact of deletion on SIFT.

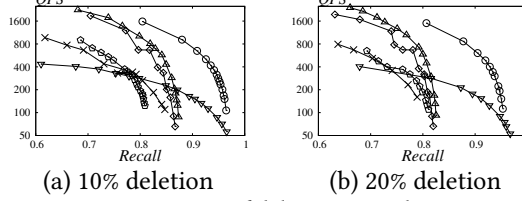


Fig. 9. Impact of deletion on Redcaps.

one million objects that involve both real-world vectors and numeric attributes are extracted to be the data objects. We generated a thousand query vectors for each dataset. For SIFT and GIST, we use query vectors provided by the datasets. For Redcaps, we create a set of one thousand query vectors by asking ChatGPT-4 to come up with queries for an image search system and embedding using CLIP. For WIT, we randomly extract one thousand vectors from the dataset as query vectors.

Competitors. We include the following methods (as we have reviewed in Sec. 2.3) in our experimental comparisons: (i) SeRF [40], (ii) SuperPostfiltering [12], (iii) iRangeGraph [38], (iv) ACORN [29], (v) Prefiltering, and (vi) Postfiltering, using HNSW as the index. Our code and more experimental details can be found in [1].

Evaluation Metrics and Parameters. Following existing benchmarks [38, 40], we use *QPS* (Queries Per Second) to measure efficiency and use *Recall* to show the accuracy. Suppose that R^* is the exact k nearest neighbors and R is the result of ANNS, the *Recall*(R) is define as $Recall(R) = \frac{|R \cap R^*|}{|R|} = \frac{|R \cap R^*|}{k}$. We set $k = 10$ for all experiments.

In addition, we report the relative error [30] to examine the quality of query results. The relative error is derived by comparing the distance of the i -th closest neighbor returned by the ANNS to the exact i -th closest neighbor for query q , and then averaging these errors across all i -th neighbors. Note that in some experiments, Posterfiltering, ACORN, and SeRF are omitted because they fail to return k results, making the calculation of relative errors undefined. In terms of construction parameters, for our method, we set $B = 3$ for the multi-way tree T . The NSW graph construction process includes two parameters M and ef_{con} . The number M is set to 16 for SIFT, 32 for GIST, Redcaps and WIT. The base ef_{con} is set to 400 for all datasets. We also conduct experiments to examine the impact of these parameters, which can be found in our technical report [1]. For SeRF, iRangeGraph and Postfiltering, the settings for the parameter M of their HNSW graph are the same as DIGRA. For ACORN, M is set to 32, M_β is set to 64, γ is set to 100 since the smallest selectivity in

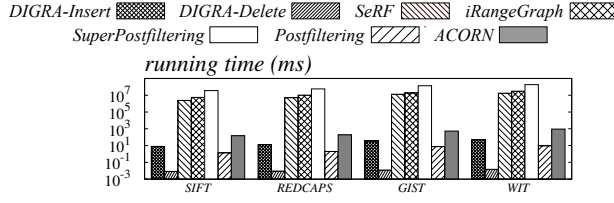


Fig. 10. Update time.

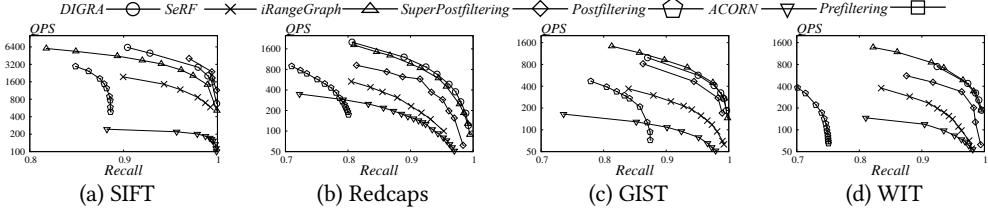


Fig. 11. QPS vs. recall (Prefiltering is omitted in Fig. 11-17 if its QPS < 50).

our experiment is 1%, as recommended in their paper. Other parameters of these methods are their default settings. As SuperPostfiltering is based on Vamana, we use its recommended parameters, i.e., $\beta = 2$, $ef = 500$, $m = 64$ for all datasets. Apart from Prefiltering, we adjust parameters during the search to control the QPS-Recall trade-off during the query phase. For SuperPostfiltering, the parameters are k and $final_multiply$, and for other methods, the parameter is ef in greedy search. The default coverage of query ranges is set to 10%, i.e., 10% of the objects falling into the range.

6.2 Experimental Comparisons

Exp 1: Impact of update. Fig. 7 shows the query performance of various methods on RedCaps after data insertion (SIFT results are shown in Fig. 1, Sec. 1). Due to space constraints, we report only SIFT and RedCaps results; similar trends for other datasets are detailed in our technical report [1]. In this set of experiments, we start with 80% or 90% of vectors from these datasets, and then add the remaining 10% or 20% vectors to the dataset. Fig. 8-9 shows the query performance of various methods after data deletion on SIFT and RedCaps. In this set of experiments, we start with all of vectors from these datasets, and then remove the 10% or 20% vectors from the dataset. For SeRF, iRangeGraph, and SuperPostFiltering, as they are static indexes, we use the index built before the update to process the range-filtered ANNS queries. We can observe that the query performance of SeRF, SuperPostFiltering, and iRangeGraph degrades significantly, as these methods are designed for static data. We can also find that as the amount of data inserted/deleted increases, the query performance of indexes designed for static data further degrades, with recall unable to exceed 0.85. Postfiltering and ACORN can achieve relatively stable accuracy through updates, but with a suboptimal QPS, as they are not designed for range-filtered ANNS. In contrast, our DIGRA maintains stable and high QPS even with dynamic data changes. As seen from the curve, our DIGRA is insensitive to data changes, showing its strong capability in handling ANNS with range filters for dynamic vector data.

Exp 2: Update efficiency. To evaluate update efficiency, we start with the full dataset, remove 10% of the vectors to test deletion efficiency, and then reinsert the 10% to measure insertion efficiency. We report the average time for each insertion and deletion operation. For static methods such as SeRF, SuperPostFiltering, and iRangeGraph, we report their index rebuild time for comparison. For ACORN and Postfiltering, we report their insertion times, since their deletions only made marks without any other changes. The update times for all methods across different datasets are shown

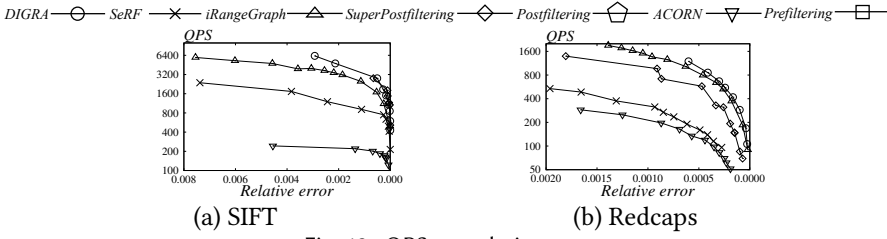


Fig. 12. QPS v.s. relative error.

in Fig. 10. The proposed method, DIGRA, demonstrates exceptional update performance, with insertion and deletion operations being over five and eight orders of magnitude faster, respectively, than static indexes that require rebuilding.

The insertion time of DIGRA is on the same order of magnitude as Postfiltering and both outperform ACORN. Additionally, DIGRA-Delete outperforms DIGRA-Insert. This is because, with randomly generated updates, most deletions only require marking objects for deletion in the ANN index of affected nodes without reconstructing the tree nodes or their ANN indices. The millisecond-level update capability of DIGRA highlights its superior performance in handling ANNS with range filters in dynamic vector data scenarios.

Exp 3: Query performance on static data. In this experiment, we demonstrate that even without changes to the dataset, our proposed method, DIGRA, remains highly efficient for range-filtered ANNS while maintaining the same level of recall. Fig. 11 shows the query performance v.s. recall for all methods on static datasets with a range coverage of 10%, indicating that 10% of the objects fulfill the range condition Q on attribute ϕ . Prefiltering is excluded due to its QPS being less than 50. Our index and query strategies, designed for dynamic updates, continue to deliver excellent performance on static datasets compared to other methods when achieving the same level of recall. Analyzing the results, SeRF shows suboptimal query performance across all datasets, primarily due to its heavy compression. While SuperPostfiltering slightly outperforms DIGRA on the SIFT dataset, it lags behind both DIGRA and iRangeGraph on other datasets. Notably, our method outperforms iRangeGraph on low-dimensional datasets like SIFT and Redcaps and performs comparably on high-dimensional datasets like GIST and WIT. This can be attributed to iRangeGraph's dynamic edge selection strategy, which incurs significant computational costs in low-dimensional spaces. In contrast, in high-dimensional spaces, the cost of distance computations becomes the dominant factor, affecting all methods similarly. In other words, our DIGRA supports efficient updates and also does so without any compromise to query efficiency—even outperforming other methods in various scenarios. This demonstrates that DIGRA effectively balances the need for dynamic updates with high query performance, making it highly suitable for both dynamic and static datasets. Postfiltering and ACORN significantly lag behind the ANNS index designed for range filtering, as their designs are not optimized for this type of query. Besides, we further report the query performance v.s. relative error for all methods in Fig. 12. We only report the results on SIFT and RedCaps datasets as the results for the other two datasets shows a similar trend and the full results can be found in our technical report [1]. As shown in Fig. 12, our DIGRA demonstrates the best trade-off between QPS and relative error across both datasets.

Exp 4: Indexing cost. Fig. 13 shows the memory usage of all methods across various datasets. Note that Prefiltering does not need an additional index, meaning its memory usage is equivalent to the input data size. SeRF incurs only a slightly higher memory cost compared to Prefiltering, significantly lower than the maximum space cost of $O(n^2M)$. This efficiency is achieved through an aggressive compression strategy but at the compromise of performance and query accuracy.

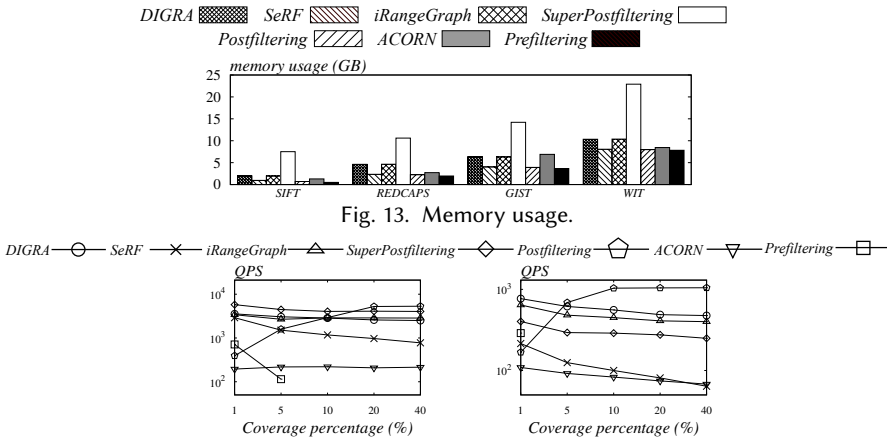


Fig. 13. Memory usage.

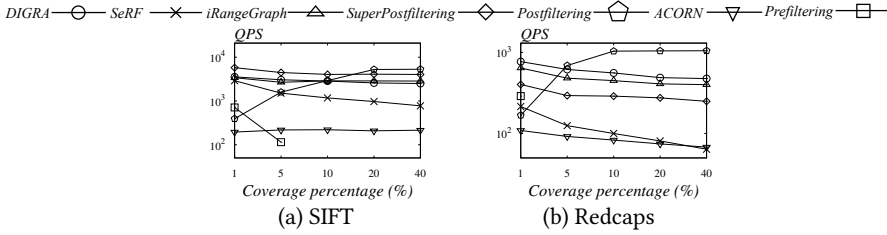


Fig. 14. Impact of query range on QPS.

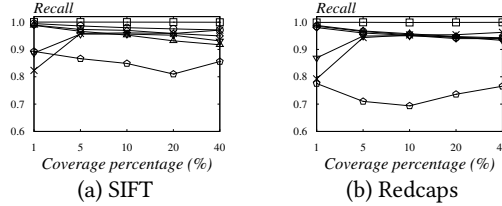


Fig. 15. Impact of query range on Recall.

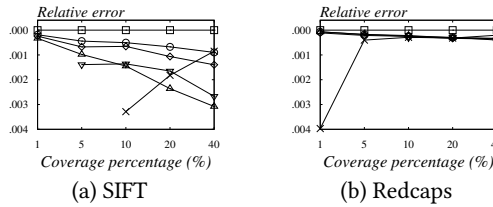


Fig. 16. Impact of query range on relative error.

Besides, SuperPostFiltering incurs excessive memory usage to achieve comparable performance as our DIGRA and iRangeGraph, requiring up to five times the size of the input data. In contrast, our DIGRA maintains a memory footprint comparable to iRangeGraph, ACORN, and Posterfiltering, while delivering outstanding query performance. This balance ensures that DIGRA achieves high efficiency without incurring high memory costs, making it a superior choice for range-filtered ANNS tasks. For indexing time, Fig. 10 has already shown the indexing time for the other methods. For DIGRA, the indexing time is comparable to that of iRangeGraph and is omitted here for brevity.

Exp 5: Impact of query range. In this set of experiment, we examine the impact of the range coverage to the query performance. Fig. 14-16 show the query performance of different methods across all datasets when varying the range filter coverage ratio. We adjust the range filter across {1%, 5%, 10%, 20%, 40%} and display the corresponding QPS, recall, and relative error. Due to limited space, we only report the results on SIFT and Redcaps datasets. The omitted results can be found in our technical report [1], which shows a similar conclusion. We can observe that DIGRA performs well, maintaining recall above 0.9, a relative error below 0.1% while ensuring high QPS. Because SeRF uses MaxLeap to compress ANN index, which loses a significant amount of information, its

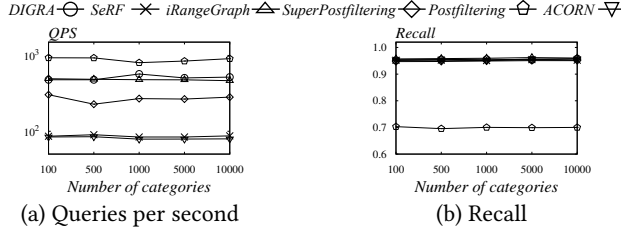


Fig. 17. Impact of the number of categories on Redcaps.

query performance is inferior to other methods, especially when the coverage ratio of range filter is less than 10%. On the SIFT dataset, SuperPostFiltering achieves the best QPS, but it consumes a large amount of space to store pre-built ANN indices as shown in Exp. 4. DIGRA gains a higher recall and smaller relative error than iRangeGraph, consistent with our observations in Exp. 3. Similar experimental results can be observed on the Redcaps datasets. Our DIGRA significantly dominates ACORN, gaining far better QPS and higher recall. For Postfiltering, it has high QPS but at the sacrifice of recalls, falling below 0.7 on Redcaps. Besides, Postfiltering is very sensitive to the coverage ratio, since it will need to visit more irrelevant objects during the search with a low coverage ratio. In contrast, our DIGRA gains stable performance across all coverage ranges. Overall, DIGRA shows stable and superior query performance across various query ranges while at the same time supporting updates.

Exp. 6: Impact of number of categories. We assess how DIGRA and competing methods handle different numbers of attribute values in Redcaps by partitioning its continuous attributes into 10,000, 5,000, 1,000, 500, or 100 buckets. Each dataset variant thus has 10,000, 5,000, 1,000, 500, or 100 categories, with each bucket holding 100, 200, 1,000, 2,000, or 10,000 objects. As shown in Fig. 17, all methods handle duplicate attributes well and are largely insensitive to the number of categories.

Exp. 7: Impact of parameters and ablation study. We conduct experiments to evaluate the impact of the parameters m and ef_{con} on query performance, indexing time, and memory usage. Additionally, we perform an ablation study on the proposed optimizations, such as α -approximate coverage. Interested readers are referred to our technical report [1] due to space limitations.

7 Conclusions

In this paper, we introduce DIGRA, an effective index that achieves a superb balance among query efficiency, update efficiency, indexing cost, and query result quality (measured by recall). Compared to existing methods, DIGRA supports dynamic updates without compromising query efficiency or increasing indexing space, making it as competitive as all current static methods. Additionally, it can process insertions in milliseconds and deletions in microseconds.

Acknowledgments

This work is supported by grants from the Research Grants Council of the Hong Kong Special Administrative Region, China (No. 14217322, No. PolyU 25201221, and No. PolyU 15205224), the Hong Kong ITC ITF (No. MRP/071/20X), the Tencent Rhino-Bird Focused Research Grant, and the Alibaba Research Intern Program.

References

- [1] 2025. Code and technical report. <https://github.com/CUHK-DBGroup/DIGRA>.
- [2] Alexandr Andoni and Piotr Indyk. 2008. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM* 51, 1 (2008), 117–122.
- [3] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya P. Razenshteyn, and Ludwig Schmidt. 2015. Practical and Optimal LSH for Angular Distance. In *NeurIPS*. 1225–1233.
- [4] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proc. VLDB Endow.* 9, 4 (2015), 288–299.
- [5] Lars Arge and Jeffrey Scott Vitter. 2003. Optimal External Memory Interval Management. *SIAM J. Comput.* 32, 6 (2003), 1488–1508.
- [6] Artem Babenko and Victor S. Lempitsky. 2015. Tree quantization for large-scale similarity search and classification. In *CVPR*. 4240–4248.
- [7] Davis W. Blalock and John V. Guttag. 2017. Bolt: Accelerated Data Mining with Fast Vector Compression. In *SIGKDD*. 727–735.
- [8] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *NeurIPS*.
- [9] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *POCG*. 253–262.
- [10] Hiroyuki Deguchi, Taro Watanabe, Yusuke Matsui, Masao Utiyama, Hideki Tanaka, and Eiichiro Sumita. 2023. Subset Retrieval Nearest Neighbor Machine Translation. In *ACL*. 174–189.
- [11] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *WWW*. 577–586.
- [12] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate nearest neighbor search with window filters. In *ICML*. Article 497, 22 pages.
- [13] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.
- [14] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *WWW*. 3406–3416.
- [15] Martin Grohe. 2020. word2vec, node2vec, graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data. In *PODS*.
- [16] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *SIGKDD*. 855–864.
- [17] Piotr Indyk and Rameez Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.
- [18] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128.
- [19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2021), 535–547.
- [20] Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *ICML*. 1188–1196.
- [21] Hui Li, Tsz Nam Chan, Man Lung Yiu, and Nikos Mamoulis. 2017. FEXIPRO: Fast and Exact Inner Product Retrieval in Recommender Systems. In *SIGMOD*. 835–850.
- [22] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [23] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Inf. Syst.* 45 (2014), 61–68.
- [24] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.
- [25] Yusuke Matsui, Ryota Hinami, and Shin'ichi Satoh. 2018. Reconfigurable Inverted Index. In *ACM Multimedia Conference on Multimedia Conference, MM*. ACM, 1715–1723.
- [26] Yuxian Meng, Xiaoya Li, Xiaoyu Zheng, Fei Wu, Xiaofei Sun, Tianwei Zhang, and Jiwei Li. 2022. Fast Nearest Neighbor Machine Translation. In *ACL*. Association for Computational Linguistics, 555–565.
- [27] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *NeurIPS* 26 (2013).
- [28] James Jie Pan, Jianguo Wang, and Guoliang Li. 2023. Survey of vector database management systems. (2023).
- [29] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

- [30] Marco Patella and Paolo Ciaccia. 2008. The many facets of approximate similarity search. In *First International Workshop on Similarity Search and Applications (sisap 2008)*. 10–21.
- [31] Mattis Paulin, Matthijs Douze, Zaïd Harchaoui, Julien Mairal, Florent Perronnin, and Cordelia Schmid. 2015. Local Convolutional Features with Unsupervised Training for Image Retrieval. In *ICCV*. 91–99.
- [32] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. Rand-NSG: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 13748–13758.
- [33] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *SIGMOD*. 2614–2627.
- [34] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978.
- [35] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proc. VLDB Endow.* 13, 12 (2020), 3152–3165.
- [36] Jiuqi Wei, Botao Peng, Xiaodong Lee, and Themis Palpanas. 2024. DET-LSH: A Locality-Sensitive Hashing Scheme with Dynamic Encoding Tree for Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 9 (2024), 2241–2254.
- [37] Paul Wohlhart and Vincent Lepetit. 2015. Learning descriptors for object recognition and 3D pose estimation. In *CVPR*. 3109–3118.
- [38] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.
- [39] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *OSDI*. 377–395.
- [40] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1 (2024), 26 pages.

Received October 2024; revised January 2025; accepted February 2025